ИНТЕЛЛЕКТУАЛЬНЫЕ СИСТЕМЫ, АНАЛИЗ ДАННЫХ, МАШИННОЕ ОБУЧЕНИЕ

УДК 004.75 ISSN 1995-5499
DOI: https://doi.org/10.17308/sait/1995-5499/2026/2/106-120
Поступила в редакцию 04.10.2025
Подписана в печать 30.06.2026

МНОГОАГЕНТНАЯ СИСТЕМА ДЛЯ АДАПТИВНОГО УПРАВЛЕНИЯ ОШИБКАМИ В СЛОЖНЫХ РАСПРЕДЕЛЕННЫХ ИНФОРМАЦИОННЫХ СРЕДАХ: МОДЕЛЬ И ЭКСПЕРИМЕНТАЛЬНОЕ ОБОСНОВАНИЕ

© 2026 М. А. Принев✉, Т. М. Леденева

Воронежский государственный университет
Университетская пл., 1, 394018 Воронеж, Российская Федерация

Аннотация. В статье решается актуальная задача автоматизации управления ошибками в сложных распределенных информационных системах. Представлена разработанная и экспериментально обоснованная модель многоагентной системы (МАС) адаптивного управления ошибками, предназначенной для поддержания работоспособности распределенных информационных систем, рассматриваемых как «черный ящик». МАС представляет собой универсальную надстройку, взаимодействующую с основной системой только через наблюдаемые точки входа. Архитектура модели включает агентов-представителей, агента компенсации, агента мониторинга, агента вычислителя и асинхронную шину с очередями сообщений, что обеспечивает распределенность, асинхронность и многопоточность обработки инцидентов. Предложен трехуровневый алгоритм выбора компенсирующих воздействий с эволюционной коррекцией их рейтингов, основанный на унификации разнородных данных о наблюдаемом состоянии объектов с помощью лингвистической модели представления приближенной информации. Экспериментальная верификация подтвердила соответствие модели заявленным критериям универсальности, эффективности, адаптивности и масштабируемости. В ходе вычислительных экспериментов на универсальном эмуляторе установлено, что разработанная модель МАС достигает доли успешных разрешений инцидентов более 85%, средняя относительная погрешность отклонения статуса от эталона составляет около 3%, а время отклика растет линейно с увеличением числа объектов. Проведено сравнение с известными альтернативными стратегиями (Always Retry, Random, Static Rules, Rating-only), выявившее статистически значимое превосходство разработанной модели по точности и времени разрешения инцидентов. Практическая значимость работы обусловлена универсальностью модели, низкой ценой интеграции и возможностью применения в широком классе распределенных систем без их модификации.

Ключевые слова: многоагентная система, поддержание работоспособности, компенсирующее воздействие, адаптивность, масштабируемость, универсальность, эффективность.

ВВЕДЕНИЕ

Современные сложные распределенные информационные системы получили широкое распространение во многих критически важных областях. Обеспечение их непрерывной работоспособности является актуальной научно-технической задачей. В таких системах инструменты управления и мониторинга играют ключевую роль, обеспечивая контроль за их состоянием. Автоматизация управления ошибками в таких системах сопряжена со значительными трудностями, так как пол-

✉ Принев Мечислав Александрович
e-mail: metshislav@rambler.ru

Контент доступен под лицензией Creative Commons Attribution 4.0 License.
The content is available under Creative Commons Attribution 4.0 License.

ная и точная аналитическая модель системы часто недоступна или нерациональна в силу ее сложности, гетерогенности и распределенного характера. Это порождает неопределенность, связанную с отсутствием полного знания о внутренней структуре и процессах основной системы (ОС).

Под управлением ошибками понимается процесс выявления, анализа и исправления ошибок [1]. Управление ошибками в системах с распределенной структурой включает стратегии (способы принятия решений) и инструменты (программные средства реализации). Примерами инструментов для мониторинга, визуализации, масштабирования и автоматического перезапуска компонентов в системах с сервисной архитектурой могут служить *Prometheus*, *Grafana*, *Kubernetes* [2–4], которые в первую очередь решают задачи мониторинга и оповещения, а не автоматического поддержания работоспособности ОС. Для автоматизированного поддержания работоспособности применяются различные стратегии: безусловный повтор (*Always Retry*) [5], случайный выбор (*Random baseline*) [6], статические продукционные правила (*Static Rules*) [7], а также адаптивные подходы. Однако большинство из них требуют детального знания архитектуры ОС, не используют накопленный опыт или плохо масштабируются. В условиях неопределенности эффективность этих стратегий снижается.

Для формализации задачи введем следующие обозначения. Пусть ОС представлена множеством объектов $Ob = \{Ob_1, ..., Ob_n\}$. Для каждого объекта Ob_i известны: эталонный статус $st_i \in R^m$; текущий статус $st1_i(1) \in R^m$, наблюдаемый в момент $t \in N$; функция принадлежности $\mu_i : R^m \to [0,1]$, оценивающая близость к эталону.

Заданы пороги, в интервале между которыми поддерживается работоспособность ОС путем назначения компенсирующего воздействия: h_{inc} — порог возникновения инцидента, h_{cr} — порог возникновения критической ошибки, где $0 < h_{cr} < h_{inc} \le 1$. Инцидент возникает при $\mu_i(st1_i(t)) \le h_{inc}$, критическая ошибка — при $\mu_i(st1_i(t)) \le h_{cr}$. Цель адаптивного управления ошибками — минимизировать ожидаемое отклонение после применения:

$$j^*(t) = \arg\min_j \mathbb{E}\left[\left\|st1_i(t+1) - st_i\right\| \mid C_j, st1_i(t)\right]. \quad (1)$$

Гипотеза исследования: ОС может рассматриваться как «черный ящик», а управление ошибками осуществляться путем косвенного адаптивного воздействия через МАС [8, 9] — универсальную надстройку, взаимодействующую с системой исключительно через наблюдаемые точки входа (объекты).

Цель статьи — представить разработанную модель МАС, алгоритмы адаптивного управления на основе эволюционной коррекции рейтингов компенсирующих воздействий и экспериментальные результаты, подтверждающие эффективность подхода.

1. МНОГОАГЕНТНАЯ СИСТЕМА УПРАВЛЕНИЯ ОШИБКАМИ

1.1. Архитектура МАС

Структурная схема разработанной модели МАС представлена на рис. 1.

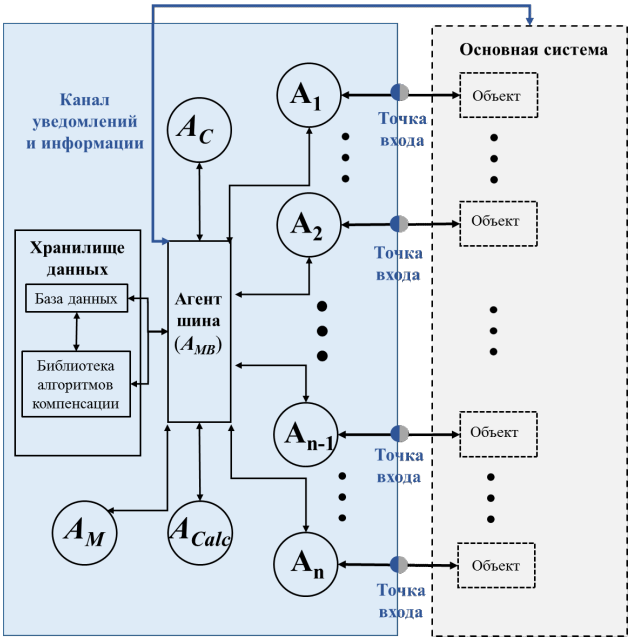


Рис. 1. Структурная схема модели МАС [Fig. 1. Block diagram of the MAS model]

Перечислим требования к модели МАС:

1. МАС должна быть универсальной надстройкой к основной системе (ОС), которая рассматривается как «черный ящик». Воздей-

ствие на ОС осуществляется косвенно, на основе наблюдаемых параметров объектов.

2. Объекты ОС должны выступать точками входа для подключения МАС. Обмен сообщениями между агентом-шиной и ОС обеспечивает канал уведомлений и информации, позволяющий получать данные о статусах и информировать ОС об инцидентах и критических ошибках.

3. Подключение МАС не должно нарушать работу ОС.

4. Агенты-представители должны быть подключены к каждому объекту и осуществлять связь между ОС и МАС.

5. МАС должна обеспечивать адаптивное управление ошибками: своевременно выявлять выход наблюдаемых показателей статусов объектов за допустимые пределы и поддерживать работоспособность ОС.

6. МАС должна обладать свойствами самоорганизации: адаптивно перестраивать внутренние параметры без внешнего вмешательства.

7. МАС должна обрабатывать разнородные данные, обеспечивать унифицированную оценку состояния объектов и выявление инцидентов.

На основе этих требований для качественной оценки разработанной модели МАС введены следующие критерии, актуальность которых подтверждена результатами вычислительных экспериментов в ходе исследования:

– $K1$ (универсальность) — соответствует требованиям 1–4: минимальные допущения, точки входа, ненарушение работы, агентное отображение.

– $K2$ (эффективность) — соответствует требованию 5: доля успешно разрешенных инцидентов, время разрешения.

– $K3$ (адаптивность) — соответствует требованию 6: скорость сходимости рейтингов, восстановление после изменений.

– $K4$ (масштабируемость) — вытекает из требований 1 и 2: линейный рост сложности, многопоточность.

В [10] представлены структурная и функциональные схемы разработанной модели МАС, входящих в нее агентов, а также дано описание процессов, лежащих в основе функционирования разработанной модели.

1.2. Информационная среда взаимодействия агентов

Для формального описания взаимодействия агентов в модели МАС введем ориентированный агентный граф $AG = (V, E)$ (рис. 2).

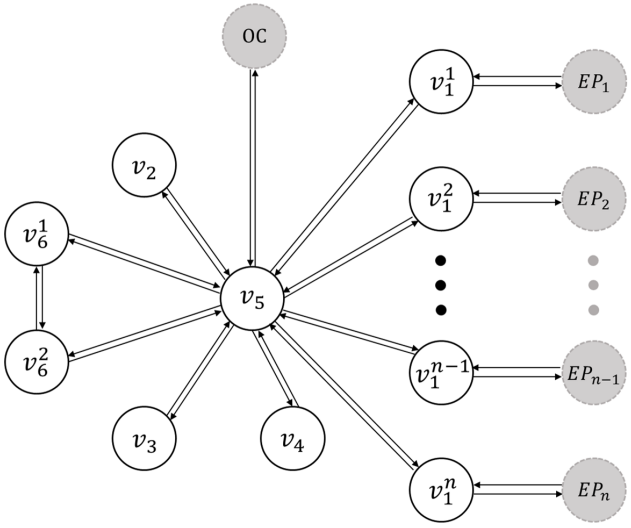


Рис. 2. Агентный граф AG
[Fig. 2. AG Agent graph]

Вершины графа AG идентифицируются следующим образом:

$v_1^1, v_1^2, \ldots, v_1^n$ — агенты-представители (A_{D_i});
v_2 — агент-компенсации (A_C);
v_3 — агент-мониторинга (A_M);
v_4 — агент-вычислитель (A_{Calc});
v_5 — агент-шина (A_{MB});
v_6^1 — база данных;
v_6^2 — библиотека алгоритмов компенсации.

Взаимодействие с основной системой (рассматриваемой как «черный ящик») и точками входа осуществляется через агентов-представителей и канал уведомлений и информации. Внешние сущности (основная система ОС и точки входа EP_i) на рисунке обозначены серым цветом и не входят в V, но участвуют во взаимодействии.

На основе описания функциональных процессов модели выделены следующие дуги (перечислены основные потоки):

(v_1^i, v_5) — передача статусов объекта (очередь Q_1); (v_5, v_2) — передача статусов аген-

ту-компенсации; (v_5, v_4) и (v_4, v_5) — обмен сообщениями при масштабировании; (v_4, v_5) — распределение вычислительных подзадач (очередь Q_2); (v_5, v_1^i) — доставка подзадач агентам-представителям; (v_1^i, v_5) — возврат результатов вычислений (очередь Q_3); (v_5, v_4) — сбор результатов вычислений; (v_4, v_5) и (v_5, v_2) — передача вектора приоритетных кластеров; (v_5, v_3) — передача текущих статусов агенту-мониторинга; (v_3, v_5) — запросы на получение статусов без воздействия; (v_5, v_1^i) — запросы статусов без воздействия; (v_2, v_5) — отправка компенсирующего воздействия; (v_5, v_1^i) — доставка компенсирующего воздействия агенту-представителю; (v_2, v_5) и (v_5, v_6^1), (v_5, v_6^2) — взаимодействие с хранилищем данных; (v_6^1, v_6^2) и (v_6^2, v_6^1) — взаимодействие базы данных и библиотеки алгоритмов компенсации; (v_3, v_5) — отправка уведомлений об инцидентах и критических ошибках; (v_5, OC) и (OC, v_5) — канал уведомлений и информации с основной системой; (v_1^i, EP_i) и (EP_i, v_1^i) — взаимодействие агентов-представителей с точками входа (вне графа).

Протокол обмена сообщениями в модели МАС основан на асинхронной передаче через очереди агента-шины с гарантией доставки. Каждое сообщение формируется в формате XML и содержит тип, идентификатор отправителя/получателя и полезную нагрузку. Для связи с основной системой используется отдельный канал уведомлений, работающий по принципу «отправил и забыл» (*fire-and-forget*), что минимизирует задержки.

2. ОСОБЕННОСТИ ОРГАНИЗАЦИИ ВЗАИМОДЕЙСТВИЯ АГЕНТОВ: АРХИТЕКТУРНЫЕ ПРИНЦИПЫ И ИХ ЭКСПЕРИМЕНТАЛЬНОЕ ОБОСНОВАНИЕ

В разработанной модели МАС взаимодействие агентов организовано на основе принципов многоагентных технологий, а именно распределенности, асинхронности и многопоточности [11]. В данном разделе экспериментально показывается, что предлагаемая организация взаимодействия (асинхронная шина, распределенные вычисления, многопоточность) наиболее эффективна по сравнению со структурными альтернативами: полносвязным взаимодействием агентов и синхронной обработкой.

2.1. Экспериментальное обоснование принципа распределенности

Для количественного обоснования преимуществ принципа распределенности, заложенного в архитектуру МАС, проведем аналитическое сравнение двух сценариев обработки данных, централизованного и распределенного, учитывая следующие параметры модели: τ — время выполнения одной вычислительной подзадачи для одного объекта (на одном агенте-исполнителе); α — стоимость декомпозиции задачи (на один объект); β — коммуникационная стоимость передачи одного сообщения (через агента-шину); γ — стоимость агрегирования результатов (на один объект); M — количество доступных агентов-исполнителей; N — число объектов. Для иллюстративных расчетов примем: $\tau = 1.0,\ \alpha = 0.1,\ \beta = 0.05,\ \gamma = 0.1$.

Рассмотрим сценарий *А* (централизованный), в котором агент вычислитель последовательно обрабатывает все N объектов. Общее время выполнения составит $T_A(N) = N(\tau + \alpha + \gamma)$.

В сценарии *Б* (распределенном) происходит декомпозиция задачи на N подзадач, которые рассылаются M доступным агентам-исполнителям. После параллельного выполнения вычислений результаты собираются и агрегируются. Время выполнения задачи можно описать следующим выражением: $T_Б(N, M) = N(\alpha + 2\beta + \gamma) + \tau[N / M]$.

Проведем анализ результатов. На рис. 3 приведены графики зависимостей $T_A(N)$ и $T_Б(N, M)$ для случаев $N = M$ (идеальный параллелизм) и $M = \text{const} = 10$.

Из графиков видно, что при $N > 10$ распределенный сценарий даже с фиксированным участвующих в распределенном решении подзадач агентов дает существенный выигрыш по времени. Коэффициент ускорения $S = T_A / T_Б$ при больших N асимптотически стремится к значению $S_{идеал} \approx \dfrac{\tau + \alpha + \gamma}{\alpha + 2\beta + \gamma}$.

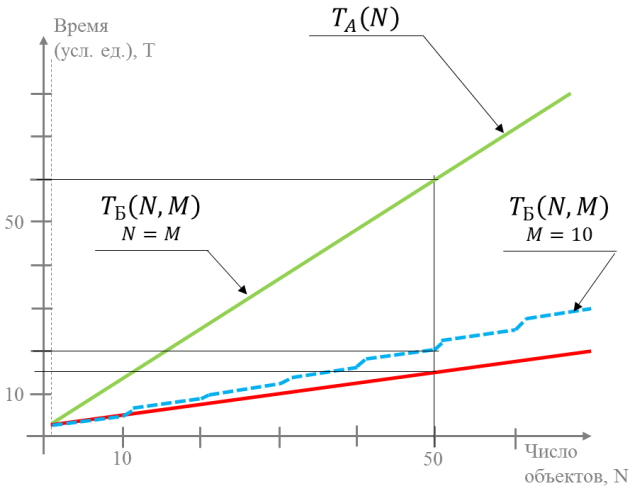


Рис. 3. Графическое сравнение сценариев
[Fig. 3. Graphical comparison of scenarios]

Для принятых числовых значений получаем $S \approx 4$, что означает четырехкратное сокращение времени выполнения по сравнению с централизованным сценарием. В результате вычислительного эксперимента обосновано, что распределенная обработка обеспечивает масштабируемость (К4), снижает требования к мощности узлов (К1) и ускоряет вычисления (К2).

Одним из ключевых архитектурных решений модели МАС является организация взаимодействия агентов через центральный компонент — агента-шину (A_{MB}). Рассмотрим два варианта организации связей между агентами МАС (без учета самой шины). Пусть общее число взаимодействующих агентов равно N (включая агентов-представителей, агента мониторинга, агента компенсации и агента вычислителя).

При полносвязной топологии количество необходимых коммуникационных дуг E_{full} определяется как число неупорядоченных пар $E_{full} = N(N-1)/2$.

Например, при $N = 50$ (типичный размер распределенной системы) число связей достигает 1225. Каждая дуга требует настройки соединения, поддержания канала и обработки коллизий. С ростом N сложность сети растет квадратично, что быстро приводит к возрастанию задержек, деградации производительности и усложнению администрирования системы, тогда как при используемая для асинхронного режима топология «звезда», обеспечивает линейный рост числа связей, а задержка при достаточной пропускной способности шины практически не возрастает, $\tau_{async} \approx \text{const}$. Уже при $N = 10$ разница становится наглядной, а при $N = 50$ полносвязный граф требует в 25 раз больше соединений. Таким образом, звездообразная топология с шиной снижает сложность сети с $O(N^2)$ до $O(N)$, что обеспечивает масштабируемость ($K4$) и упрощает интеграцию ($K1$).

2.2. Экспериментальное обоснование принципа асинхронности

Использование топологии «звезда» в архитектуре модели МАС решает проблему числа связей, но не снимает вопроса о блокировках и задержках при пиковых нагрузках. Асинхронный обмен через очереди, реализованный в агенте-шине, обеспечивает: отсутствие блокировок (отправитель не ждет ответа), буферизацию пиковых нагрузок и упрощение логики агентов (достаточно знать адрес шины).

Рассмотрим агента-шину как одноканальную систему массового обслуживания (СМО) с бесконечной очередью [12, 13]. Входящий поток сообщений формируется всеми N агентами системы (агентами-представителями и служебными агентами). Примем следующие допущения и параметры модели: λ (интенсивность поступления сообщений от одного агента, среднее число сообщений, отправляемых агентом в единицу времени), $\Lambda = N \cdot \lambda$ (суммарная интенсивность входящего потока на шину), μ_{bus} (интенсивность обслуживания шины, среднее число сообщений, которое шина способна обработать — принять, маршрутизировать и доставить адресату — в единицу времени), $\rho = \Lambda / \mu_{bus} < 1$ (коэффициент загрузки, условие стабильности).

Для приближенной оценки задержек воспользуемся классической формулой Литтла для системы M/M/1 [14], которая, несмотря на упрощающие предположения (пуассоновский входящий поток и экспоненциальное время обслуживания), дает качественно верную картину и широко применяется для анализа коммуникационных сетей $W = \dfrac{\rho}{\mu_{bus}(1-\rho)}$ [15].

На рис. 4 представлен качественный график зависимости среднего времени ожидания W от числа агентов N, построенный для фиксированных значений λ и μ_{bus}.

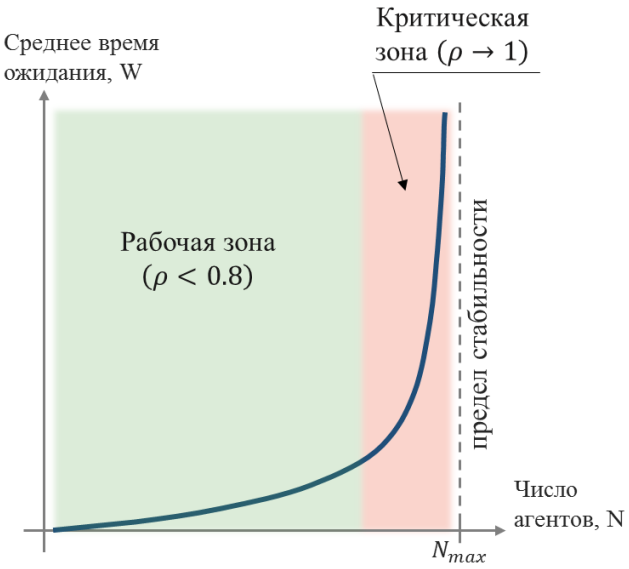


Рис. 4. Достижение предела стабильности [Fig. 4. Reaching the limit of stability]

Кривая наглядно демонстрирует наличие «безопасной» зоны (пологого линейного роста) и «критической» зоны вблизи порога стабильности. Вычислительный эксперимент позволяет сделать вывод, что пока нагрузка не превышает 80 % от пропускной способности шины ($\rho < 0.8$), задержки растут практически линейно, обеспечивая масштабируемость ($K4$). Отсутствие блокировок упрощает архитектуру ($K1$).

2.3. Экспериментальное обоснование принципа многопоточности

Покажем, что использование принципа многопоточности — параллельная обработка инцидентов в разработанной модели МАС сокращает общее время реакции системы. Ускорение реакции модели по отношению к однопоточной обработке определяется следующей формулой: $S(p) = T(1)/T(p)$. С учетом накладных расходов имеем $T(p) = (1/p)\sum t_{proc_i} + t_{comm(p)}$, где параметры модели: p — число параллельных потоков; $T(1)$ — время однопоточной обработки; $T(p)$ — (время многопоточной обработки); $t_{comm(p)}$ — коммуникационная стоимость (затраты на синхронизацию через шину); t_{proc_i} — время выполнения подзадачи для i-го объекта. Расчеты выполнены при $t_{\text{proc}_i}/t_{\text{comm}}(1) = 10$ что типично для задач, где вычисления доминируют над обменом. На рис. 5 сплошная линия — полученное ускорение, пунктир — идеальное $S(p) = p$. Ускорение близко к линейному в диапазоне $p < 4$. При $p > 4$ рост замедляется, так как накладные расходы на взаимодействие агентов начинают преобладать над временем вычислений. Таким образом, многопоточная архитектура обеспечивает рост ускорения, близкий к линейному в рабочем диапазоне p, что подтверждает эффективность использования ресурсов ($K2$) и масштабируемость ($K4$). Ограничением является пропускная способность шины.

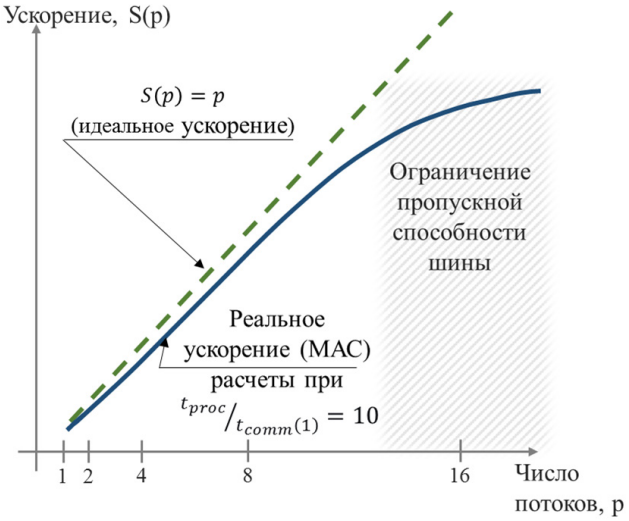


Рис. 5. Пропускная способность шины [Fig. 5. Bus capacity]

3. УПРАВЛЕНИЕ НА ОСНОВЕ КОМПЕНСИРУЮЩИХ ВОЗДЕЙСТВИЙ

Разработанная модель МАС выполняет следующие задачи:

– сбор и обработка статусов объектов, выполняется агентами-представителями и агентом-вычислителем;

– выявление инцидентов и выбор компенсирующего воздействия (КВ), выполняется агентом-компенсации;

– мониторинг результата, отмена воздействий, уведомление ОС, выполняется агентом-мониторинга;

– динамическая адаптация рейтингов, осуществляется в хранилище данных (ХД), по специальному алгоритму.

В ходе исследования была разработана программная реализация алгоритма работы с КВ, позволяющая эмулировать процессы, происходящие в ХД и осуществлять интеллектуальный выбор КВ при возникновении инцидентов в ОС.

Рассмотрим подробнее процесс работы с КВ. Он включает в себя трехуровневый отбор КВ для разрешения инцидента из библиотеки алгоритмов компенсации и интеллектуальную коррекцию рейтинга компенсирующих воздействий в базе данных. Рейтинг R, представляет собой относительную прогностическую оценку вероятности успешного применения КВ. Первоначально рейтинг правил задается произвольно или в соответствии с экспертной оценкой, которая является стартовой количественной величиной, оценивающей успешное применение КВ, и используется для уменьшения времени оптимизации библиотеки алгоритмов компенсации. При холодном старте начальные рейтинги КВ задаются с равным интервалом в единицу, что упрощает ранжирование на начальном этапе. По результатам применения КВ производится коррекция рейтинга. Чем больше значение приобретенного рейтинга, тем более универсальным, эффективным и востребованным является КВ. Коррекция рейтинга происходит на величину, меньшую единицы, поэтому изменение значения рейтинга, позволяющее нарушить первоначальное распределение правил, имеет эволюционный эффект и происходит после некоторого статистического накопления опыта практического применения. Величина коррекции рейтинга зависит от значения изменений показателей текущего статуса объекта, что также повышает оценку эффективности назначения КВ.

Ключевые свойства адаптивного изменения рейтинга КВ: *накопление* — для смены лидера требуется устойчивое статистическое преимущество; *адаптация* — после обработки серии инцидентов рейтинг оптимального правила (или правил) возрастает, а неэффективных — падает.

Покажем в ходе эксперимента на эмуляторе, обработав 50 инцидентов с библиотекой из пяти КВ, что механизм адаптивного обновления рейтинга обеспечивает сходимость к оптимальной стратегии без резких скачков, за счет малой величины коррекции и эффекта статистического накопления.

Адаптивная коррекция рейтинга определяется следующей формулой $R_j(t+1) = R_j(t) + \eta \cdot \Delta\mu$, где параметры модели: $\Delta\mu = \mu(st2_i) - \mu(st1_i)$ — изменение значения статуса к норме $(|\Delta\mu| < 1)$; η — коэффициент скорости адаптации $(0 < \eta \le 1)$.

На полученных графиках (рис. 6) можно наблюдать, что первоначальная иерархия правил претерпела изменения, причем адаптация рейтинга, за счет свойства накопления, показала постепенную, но стабильную коррекцию расположения КВ относительно друг друга.

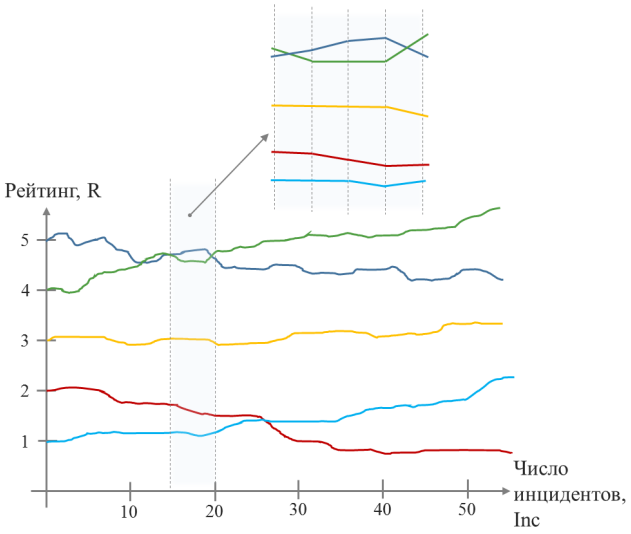


Рис. 6. Адаптивная коррекция рейтинга [Fig. 6. Adaptive rating correction]

Таким образом, экспериментально подтверждено, что механизм адаптации с малой величиной коррекции $(|\Delta\mu| < 1)$ обеспечивает устойчивую сходимость к оптимальной стратегии, подтверждая адаптивность $(K3)$ и эффективность $(K2)$ системы. Экспертная инициализация рейтингов гарантирует работоспособность уже на этапе холодного старта $(K1)$.

В ХД происходит трехуровневый подбор КВ для поддержания работоспособности ОС. Многопоточная обработка данных позволяет

при необходимости осуществлять несколько итераций по выбору КВ, укладываясь в заданный лимит времени. Рассмотрим принципы выбора КВ. Каждому КВ присущи текущее значение рейтинга, не менее одного значения терма из шкалы

$$Scale_{Situ} = \begin{cases} S_1 = угрожающее, \\ S_2 = опасное, \\ S_3 = тревожное, \end{cases}$$

значение кластера (группа КВ в библиотеке алгоритмов компенсации, объединенная по принципу схожести существенных атрибутов объекта). Метка с принадлежностью к какому-либо кластеру присваивается КВ при настройке и подключении МАС к ОС. На первом уровне выбирается КВ с наивысшим текущим рейтингом, так называемое, «золотое правило». Со временем, эволюционная коррекция рейтинга и самоорганизация библиотеки алгоритмов компенсации приведет к тому, что КВ с наивысшим рейтингом станет статистически самым универсальным и эффективным из имеющихся, к тому же будет учитывать самые часто повторяющиеся ошибки основной системы. Если назначение КВ, выбранного по рейтингу, не привело к разрешению инцидента, то на втором уровне выбор КВ осуществляется в рамках определенного количества итераций по рейтингу и присущему ему значению терма, иначе идет переход на третий уровень, где КВ выбирается в соответствии с рейтингом и компонентами вектора приоритетных кластеров, который определяется сразу при подключении объекта ОС к МАС и зависит от атрибутов, характеризующих объект. Для работы на третьем уровне назначается количество итераций для отбора КВ в каждом из кластеров, приоритетных для объекта. Работа ХД продолжается до разрешения инцидента или до истечения фиксированного времени t_{cr} (предела времени на обработку инцидента). Изменение рейтинга КВ вычисляется после каждой итерации, но коррекция в базу данных вносится только по окончании обработки инцидента (при $t = t_{cr}$).

Покажем в ходе эксперимента на эмуляторе, обработав 50 инцидентов, как адаптация рейтингов меняет динамику использования трехуровневой стратегии.

Доля срабатываний i-го уровня: $D(L_i) = N_{L_i} / (N_{L_1} + N_{L_2} + N_{L_3})$, где параметры модели: $N_{L_1}, N_{L_2}, N_{L_3}$ — количество срабатываний каждого уровня за период наблюдений; L_1 — «золотое правило» (уровень 1); L_2 — поиск по термам (уровень 2); L_3 — поиск по вектору приоритетных кластеров (уровень 3).

На полученных графиках, рис. 7, можно наблюдать, что в зоне холодного старта распределение срабатывания КВ по уровням было практически одинаково, расхождение в долях срабатывания уровней начало проявляться в переходном периоде, в фазе активной адаптации расхождение имело максимальное ускорение, а в фазе стабильной адаптации расхождение долей срабатывания трехуровневой стратегии стало практически неизменным, показав явное преимущество первого уровня.

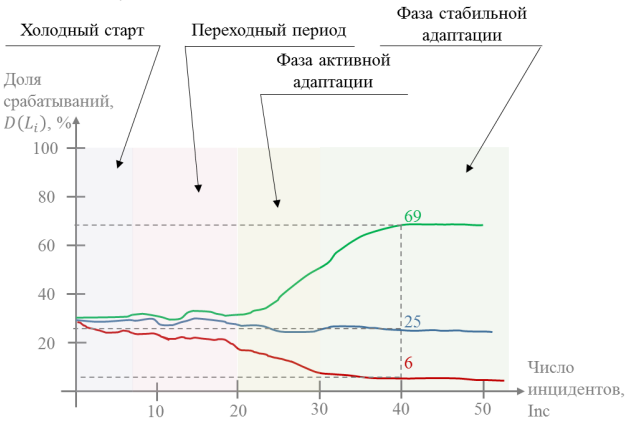


Рис. 7. Динамика трехуровневой стратегии [Fig. 7. Dynamics of the three-level strategy]

Рассмотрим динамику адаптации модели, исследовав долю оптимальных выборов при изменении условий. На эмуляторе было обработано 100 инцидентов, сценарий эксперимента предусматривал изменение условий (введение новых объектов) после обработки 40 инцидентов.

Доля верных выборов (скользящее среднее) есть $Acc(m) = \dfrac{N_{correct}(m)}{m}$. Падение точности после изменения условий есть $Acc = Acc_{be\ fore} - Acc_{after}$, где параметры модели: m — номер инцидента; $Acc(m)$ — точность выбора; m_{change} — момент изменения

условий; m_{rec} — число инцидентов до возврата $Acc(m)$ на уровень > 0.85.

Полученный график (рис. 8) показывает, что в зоне стабильной адаптации доля верных выборов не опускается ниже 85 %, изменение условий приводит к кратковременному спаду доли верных выборов, минимальное значение Acc при спаде больше, чем при холодном старте.

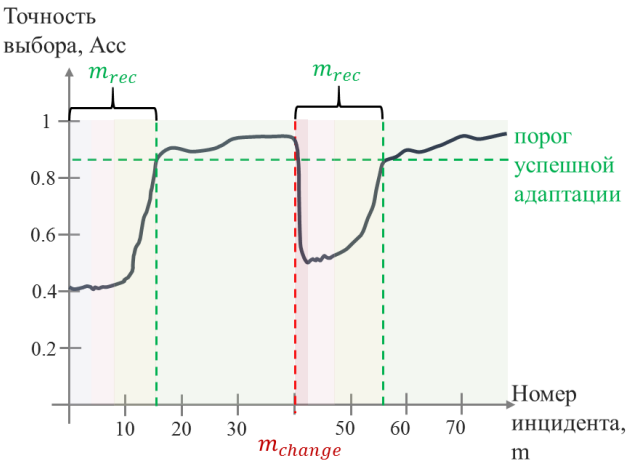


Рис. 8. Динамика адаптации модели
[Fig.8. Dynamics of model adaptation]

Таким образом, система достигает больше 85 % доли верных выборов после завершения фазы активной адаптации, подтверждая эффективность $(K2)$. При резком изменении условий точность временно падает, но система восстанавливается за конечное число итераций, демонстрируя адаптивность $(K3)$.

Было экспериментально подтверждено, что разработанная модель МАС решает формальную задачу (1) минимизации ожидаемого отклонения показателей статусов объекта от эталонных. В ходе эксперимента были использованы следующие параметры и метрика модели: для каждого инцидента фиксировалось эмпирически оптимальное воздействие $C_{opt}(t)$, определяемое полным перебором на эмуляторе; вычислялась относительная погрешность отклонения статуса

$$\delta = \frac{\left\| st1_i(t) - st_i \right\|_{факт} - \left\| st1_i(t) - st_i \right\|_{опт}}{\left\| st1_i(t) - st_i \right\|_{опт} + \varepsilon},$$

где $\| \cdot \|$ — норма (например, евклидова), st_i — эталонный статус, $\varepsilon = 10^{-6}$ — малое положительное число, вводимое для предотвращения деления на ноль.

Результаты эксперимента представлены на рис. 9.

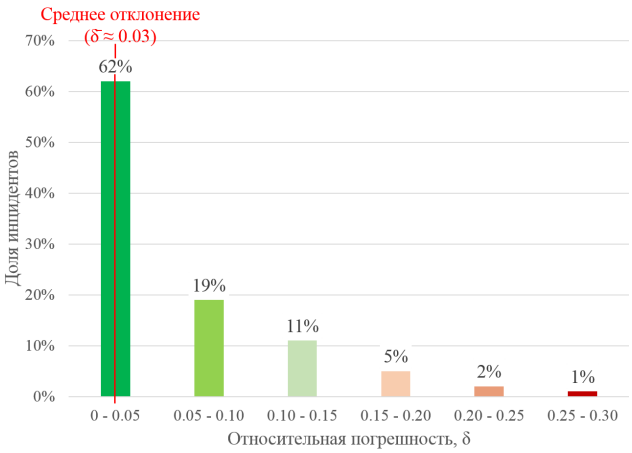


Рис. 9. Распределение относительной погрешности
[Fig. 9. Distribution of relative error]

Вывод: экспериментально подтверждено, что разработанная модель МАС минимизирует отклонение статуса от эталона: средняя относительная погрешность $\delta \approx 0,03$. Это доказывает, что предложенный алгоритм решает формальную задачу, обеспечивая эффективность $(K2)$ и адаптивность $(K3)$.

4. СРАВНЕНИЕ РАЗРАБОТАННОЙ МОДЕЛИ МАС С АЛЬТЕРНАТИВНЫМИ СТРАТЕГИЯМИ

Для оценки сравнительного преимущества разработанной модели МАС был проведен вычислительный эксперимент, в котором она сопоставлялась с рядом альтернативных стратегий управления ошибками. В качестве оппонентов рассматривались как известные инженерные и академические подходы — безусловный повтор (*Always Retry* [5]), случайный выбор (*Random baseline* [6]) и жесткие продукционные правила (*Static Rules* [7]), так и упрощенные конфигурации самой МАС, полученные путем отключения ее ключевых механизмов: использование только рейтинга (*Rating-only*) и только сигнализации без компенсации (*Alarm-only*). Последние соответствуют методологии абляционного исследования (*ablation study* [16]) и позволяют количественно оценить вклад каждого модуля системы. Сравнительная характеристика мо-

делей по набору ключевых функциональных свойств представлена в табл. 1.

Как видно из таблицы, только разработанная модель MAC обладает полным набором ключевых свойств, объединяя адаптивность, контекстный выбор и низкие требования к системе. Это подтверждает ее универсальность ($K1$), эффективность ($K2$) и адаптивность ($K3$) уже на уровне архитектуры.

Покажем, что в фазе стабильной адаптации система достигает высокой доли оптимальных выборов и способна оптимально быстро разрешать возникающие инциденты. В ходе эксперимента были эмулированы свойства каждой из сравниваемых стратегий со следующими параметрами: данные собраны после фазы активной адаптации (50+ инцидентов); проведено усреднение по 30 прогонам.

Доля верных выборов вычисляется по формуле $Acc = N_{correct} / N_{total}$. Нормированное время определяется как $T_{norm} = T_{model} / T_{MAC}$, где T_{model} — среднее время разрешения инцидента для модели с альтернативной стратегией, а T_{MAC} — среднее время разрешения для модели MAC (принято за единицу).

На рис. 10 представлена сравнительная диаграмма, показывающая результаты эксперимента для сравниваемых стратегий.

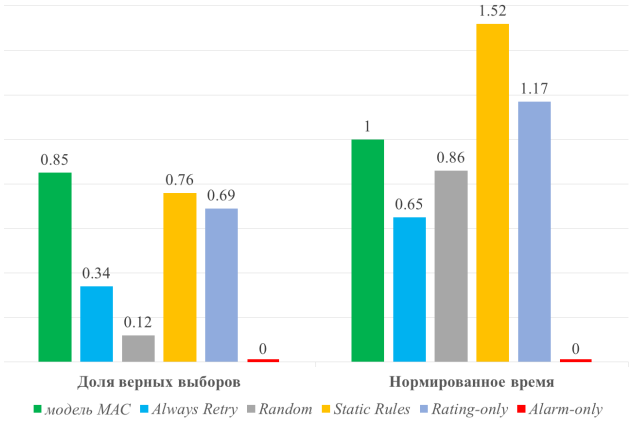


Рис. 10. Сравнение стратегий
Fig. 10. Comparison of strategies

Полученные экспериментальные значения для альтернативных стратегий Static Rules и Rating-only отклонились от теоретического прогноза, основанного на вероятностно-временной модели. Для стратегии Rating-only прогнозируемая относительная длительность (около 0,8 ед.) оказалась ниже измеренной (1,17 ед.), что объясняется необходимостью повторения итераций вплоть до исчерпания временного лимита — в отличие от разработанной модели, где после первой неудачи управление передается второму уровню. Для стратегии Static Rules зафиксировано сокращение относительного времени по сравнению с предсказанным (~2,2–2,5 ед.) до 1,52 ед., одна-

Таблица 1. Сравнительная характеристика моделей
[Table 1.Comparative characteristics of the models]

Модель	Суть стратегии	Адаптивность	Учет контекста	Масштабируемость	Универсальность	Обучаемость
Модель MAC	Трехуровневый выбор с адаптацией рейтингов	✓	✓	✓	✓	✓
Always Retry	Повтор последнего действия	✗	✗	✓	✓	✗
Random	Случайный выбор	✗	✗	✓	✓	✗
Static Rules	Жесткие правила	✗	✓ (стат.)	✗	✗	✗
Rating-only	Только рейтинг	✓	✗	✓	✓	✓
Alarm-only	Сигнализация без компенсации	✗	✗	✓	✗	✗

ко эмпирически полученное уменьшение T_{model} сопровождалось падением точности, что позволяет предполагать рост доли принудительных остановок по тайм-ауту. Результаты эксперимента выявили и эмпирически подтвердили неочевидное при расчетах преимущество разработанной модели, вызванное комплексным адаптивным подходом к выбору КВ.

По результату эксперимента можно сделать следующий общий вывод: разработанная модель МАС достигает максимальной доли верных выборов (~ 85 %), значительно превосходя модели с альтернативными стратегиями. При этом время разрешения остается сопоставимым с быстрыми, но неточными стратегиями. Это подтверждает эффективность ($K2$) и адаптивность ($K3$) системы в условиях неопределенности

В ходе исследования было неоднократно подтверждено выполнение критериев качества разработанной модели МАС: универсальность, $K1$, эффективность, $K2$, адаптивность, $K3$ и масштабируемость, $K4$. Итоговая визуализация, отражающая подтверждение критериев качества модели, выраженное в баллах, начисляемых каждым подтверждением критерия в ходе вычислительных экспериментов, а также баллов, полученных моделью в ходе сравнения с альтернативными стратегиями, представлена на рис. 11.

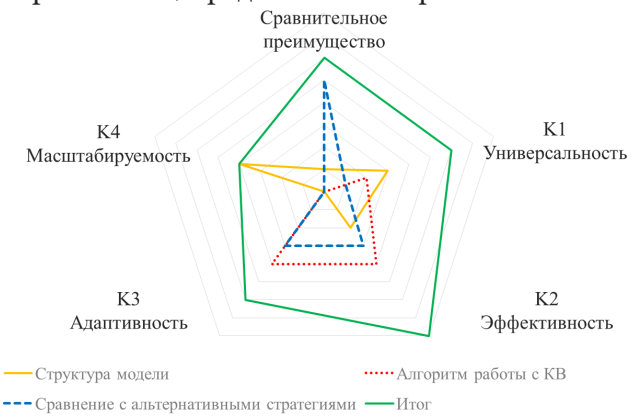


Рис. 11. Итоговая визуализация
[Fig. 11. Final visualization]

Итоговые выводы: разработанная модель МАС соответствует всем заданным критериям качества, с наибольшим перевесом критерия $K2$ (эффективность); структура модели имеет весомое преимущество по критериям $K1$ (универсальность) и $K4$ (масштабируемость); алгоритм работы с КВ показал сбалансированный вклад по всем критериям с небольшим перевесом критерия $K3$ (адаптивность); Сравнительный анализ показал приоритетное преимущество модели по критериям $K3$ (адаптивность) и $K2$ (эффективность.)

В целом, разработанная модель МАС показала свое превосходство над приведенными альтернативными стратегиями управления ошибками.

ЗАКЛЮЧЕНИЕ

В ходе исследования решена актуальная научно-техническая задача разработки и экспериментального обоснования модели МАС, предназначенная для поддержания работоспособности распределенных информационных систем. Получены следующие научные результаты:

1. Разработана модель МАС, функционирующая как универсальная надстройка без доступа к внутренней логике ОС и не требующая модификации целевой распределенной системы.

2. Предложена архитектура на принципах МАС, обеспечившая четырехкратное ускорение вычислений, линейную масштабируемость и 25-кратное сокращение связей.

3. Разработан трехуровневый алгоритм выбора компенсирующих воздействий, обеспечивающий адаптацию модели без применения методов машинного обучения.

4. Предложен способ унификации разнородных данных на основе нечетких лингвистических шкал, позволяющий выявлять отклонения без точной математической модели объекта.

5. Создана программная реализация алгоритма работы с КВ, отличающаяся универсальностью и обеспечивающая экспериментальное сравнение стратегий управления ошибками в режиме эмуляции.

Практическая значимость полученных результатов определяется: универсальностью модели (интеграция без изменений архитектуры ОС); низкой ценой интеграции (достаточно наблюдаемых точек входа и возможности

программного воздействия); возможностью применения в системах с большим числом объектов без экспоненциального роста времени отклика (линейная масштабируемость); уменьшением нагрузки на ОС за счет эволюционной коррекции рейтингов, сокращающей итерации поиска КВ по мере адаптации.

Таким образом, разработанная модель МАС является универсальным, эффективным, адаптивным и масштабируемым решением для автоматизации управления ошибками в сложных распределенных системах.

КОНФЛИКТ ИНТЕРЕСОВ

Авторы декларируют отсутствие явных и потенциальных конфликтов интересов, связанных с публикацией настоящей статьи.

СПИСОК ЛИТЕРАТУРЫ

1. *Avizienis A., Laprie J.-C., Randell B., Landwehr C.* Basic concepts and taxonomy of dependable and secure computing // IEEE Transactions on Dependable and Secure Computing. – 2004. – Vol. 1, No 1. – P. 11–33. – DOI: 10.1109/TDSC.2004.2.

2. *Quentin L.* Scalability Evaluation of Prometheus for HPC Monitoring [Электронный ресурс] / L. Quentin // Joint Laboratory for System Evaluation (JLSE) Technical Reports. – 2024. – Режим доступа: http://vi4io.org (дата обращения: 27.04.2026).

3. *Ларионов В. С., Дунин И. В.* Анализ работы веб-серверов с использованием пакета grafana // Форум молодых ученых. – 2017. – № 11(15). – С. 544–549. – EDN YOBQJM.

4. *Фирсов Н. А.* Мониторинг состояния инфраструктуры при помощи Prometheus & Grafana // Современные тенденции и инновации в науке и производстве : Материалы XII Международной научно-практической конференции, Междуреченск, 26 апреля 2023 года / Редколлегия: Т. Н. Гвоздкова (отв. редактор), С. О. Марков [и др.]. – Междуреченск : Кузбасский государственный технический университет имени Т.Ф. Горбачева, 2023. – С. 461.1–461.3. – EDN RVBADY.

5. Microsoft Docs. Retry pattern [Электронный ресурс] // Microsoft Azure Architecture Center. – Режим доступа: https://learn.microsoft.com/en-us/azure/architecture/patterns/retry (дата обращения: 17.05.2026).

6. *Sutton R. S., Barto A. G.* Reinforcement learning: An introduction. – Second edition. – Cambridge, Massachusetts : The MIT Press, 2018. – 552 p.

7. *Hayes-Roth F., Waterman D. A., Lenat D. B. (eds.)* Building expert systems. – Reading, Mass. : Addison-Wesley Pub. Co., 1983. – 444 p.

8. *Леденева Т. М., Принев М. А.* Многоагентные системы: модели агентов и постановки задач, обеспечивающие их характерные свойства (часть 1) // Вестник Воронежского государственного университета. Серия: Системный анализ и информационные технологии. – 2024. – № 3. – С. 50–73. – DOI 10.17308/sait/1995-5499/2024/3/50-73. – EDN GHJMUH.

9. *Принев М. А., Леденева Т. М., Гаршина В. В.* Многоагентные системы: обзор современных подходов к моделированию и проектированию (Часть 2) // Вестник Воронежского государственного университета. Серия: Системный анализ и информационные технологии. – 2024. – № 4. – С. 167–190. – DOI 10.17308/sait/1995-5499/2024/4/167-190. – EDN KJBPCG.

10. *Prinev M. A.* Implementation of Fuzzy Error Management in Systems with a Service Architecture Using Multi-Agent Technologies // 2025 7th International Conference on Control Systems, Mathematical Modeling, Automation and Energy Efficiency (SUMMA). – Lipetsk, Russian Federation, 2025. – P. 450–454. – DOI 10.1109/SUMMA68668.2025.11302295.

11. *Хоп Г., Вульф Б.* Шаблоны интеграции корпоративных приложений : пер. с англ. – М. : ООО «И.Д. Вильямс», 2007. – 672 с. – EDN QMQSTX.

12. *Клейнрок Л.* Вычислительные системы с очередями / пер. с англ. под ред. Б. С. Цыбакова. – М. : Мир, 1979. – 600 с.

13. *Гнеденко Б. В., Коваленко И. Н.* Введение в теорию массового обслуживания. – 2-е изд., перераб. и доп. – М. : Наука, 1987. – 336 с.

14. *Little J. D. C.* A Proof for the Queuing Formula: L = λW // Operations Research. – 1961. – Vol. 9, No 3. – P. 383–387.

15. *Gross D., Harris C. M.* Fundamentals of Queueing Theory. – 3rd ed. – Chichester : Wiley, 1998. – 439 p.

16. *Meyes R. [et al.]* Under the Hood of Neural Networks: Characterizing Learned Representations by Functional Neuron Populations and Network Ablations [Электронный ресурс] // arXiv preprint. – 2020. – arXiv:2004.01254. – Режим доступа: https://arxiv.org/abs/2004.01254 (дата обращения: 25.04.2026).

Принев Мечислав Александрович — аспирант 3-го курса кафедры вычислительной математики и прикладных информационных технологий Воронежского государственного университета.
E-mail: metshislav@rambler.ru
ORCID iD: https://orcid.org/0009-0004-9704-6896

Леденева Татьяна Михайловна — д-р техн. наук, проф., зав. кафедрой вычислительной математики и прикладных информационных технологий Воронежского государственного университета.
E-mail: ledeneva-tm@yandex.ru
ORCID iD: https://orcid.org/ 0000-0002-3944-2266

DOI: https://doi.org/10.17308/sait/1995-5499/2026/2/106-120 ISSN 1995-5499
Received 04.10.2025
Accepted 30.06.2026

MULTI-AGENT SYSTEM FOR ADAPTIVE ERROR MANAGEMENT IN COMPLEX DISTRIBUTED INFORMATION ENVIRONMENTS: MODEL AND EXPERIMENTAL JUSTIFICATION

© 2026 M. A. Prinev✉, T. M. Ledeneva

Voronezh State University
1, Universitetskaya Square, 394018 Voronezh, Russian Federation

Annotation. The article deals with automating error management in complex distributed information systems. It introduces a validated model of a multi-agent system (MAS) for adaptive error control, designed to maintain the functionality of distributed systems treated as a «black box.» The MAS interacts with the primary system through observable entry points. Its architecture includes agents for representation, compensation, monitoring, computation, and an asynchronous bus with message queues, ensuring distributed and efficient incident handling. A three-level algorithm with evolutionary correction of ratings selects compensatory actions, unifying heterogeneous data via a linguistic model of approximate information representation. Experimental verification confirmed the model's universality, efficiency, adaptability, and scalability. Emulator tests showed the MAS achieves over 85% success in resolving incidents, with minimal error and linear response time growth. Comparisons with strategies like Always Retry, Random, Static Rules, and Rating-only demonstrated the model's superior accuracy and resolution time. Its practical significance lies in its versatility, low integration cost, and broad applicability without system modifications.
Keywords: multi-agent system, performance maintenance, compensating effect, adaptability, scalability, versatility, efficiency.

✉ Prinev Mechislav A.
e-mail: metshislav@rambler.ru

CONFLICT OF INTEREST

The authors declare the absence of obvious and potential conflicts of interest related to the publication of this article

RFERENCES

1. *Avizienis A., Laprie J.-C., Randell B. and Landwehr C.* (2004) Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*. Vol. 1, No. 1. P. 11–33. DOI: 10.1109/TDSC.2004.2.

2. *Quentin L.* (2024) Scalability Evaluation of Prometheus for HPC Monitoring. Joint Laboratory for System Evaluation (JLSE) Technical Reports. Available at: http://vi4io.org (accessed: 27.04.2026).

3. *Larionov V. S. and Dunin I. V.* (2017) Analysis of web server operation using the grafana package. *Forum of Young Scientists*. No 11(15). P. 544–549. EDN YOBQJM.

4. *Firsov N. A.* (2023) Monitoring of infrastructure state using Prometheus & Grafana. *Modern trends and innovations in science and production: Proceedings of the XII International Scientific and Practical Conference*, Mezhdurechensk, April 26, 2023 (eds. Gvozdkova T. N., Markov S. O. [et al.]). Mezhdurechensk : T. F. Gorbachev Kuzbass State Technical University. P. 461.1–461.3. EDN RVBADY.

5. Microsoft Docs (2026) Retry pattern. Microsoft Azure Architecture Center. Available at: https://learn.microsoft.com/en-us/azure/architecture/patterns/retry (accessed: 17.05.2026).

6. *Sutton R. S. and Barto A. G.* (2018) Reinforcement learning: An introduction. Second edition. Cambridge, Massachusetts: The MIT Press. 552 p.

7. *Hayes-Roth F., Waterman D. A. and Lenat D. B.* (eds.) (1983) Building expert systems. Reading, Mass.: Addison-Wesley Pub. Co. 444 p.

8. *Ledeneva T. M. and Prinev M. A.* (2024) Multi-agent systems: agent models and problem statements ensuring their characteristic properties (part 1). *Bulletin of Voronezh State University. Series: System Analysis and Information Technologies*. No 3. P. 50–73. DOI: 10.17308/sait/1995-5499/2024/3/50-73. EDN GHJMUH.

9. *Prinev M. A., Ledeneva T. M. and Garshina V. V.* (2024) Multi-agent systems: a review of modern approaches to modeling and design (part 2). *Bulletin of Voronezh State University. Series: System Analysis and Information Technologies*. No 4. P. 167–190. DOI: 10.17308/sait/1995-5499/2024/4/167-190. EDN KJBPCG.

10. *Prinev M. A.* (2025) Implementation of Fuzzy Error Management in Systems with a Service Architecture Using Multi-Agent Technologies. *2025 7th International Conference on Control Systems, Mathematical Modeling, Automation and Energy Efficiency (SUMMA)*. Lipetsk, Russian Federation. P. 450–454. DOI: 10.1109/SUMMA68668.2025.11302295.

11. *Hop G. and Woolf B.* (2007) Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Moscow : I. D. Williams LLC. 672 p. (Russian translation). EDN QMQSTX.

12. *Kleinrock L.* (1979) Queueing Systems. Moscow: Mir. 600 p. (Russian translation, ed. by B. S. Tsybakov).

13. *Gnedenko B. V. and Kovalenko I. N.* (1987) Introduction to Queueing Theory. 2nd ed., rev. and suppl. Moscow : Nauka. 336 p.

14. *Little J. D. C.* (1961) A Proof for the Queuing Formula: L = λW. *Operations Research*. Vol. 9, No. 3. P. 383–387.

15. *Gross D. and Harris C. M.* (1998) Fundamentals of Queueing Theory. 3rd ed. Chichester : Wiley. 439 p.

16. *Meyes R. [et al.]* (2020) Under the Hood of Neural Networks: Characterizing Learned Representations by Functional Neuron Populations and Network Ablations. *arXiv preprint*. arXiv:2004.01254. Available at: https://arxiv.org/abs/2004.01254 (accessed: 25.04.2026).

Prinev Mechislav A. — 3nd year graduate student, Department of Computational Mathematics and Applied Information Technologies, Faculty of Applied Mathematics, Informatics and Mechanics, Voronezh State University.
E-mail: metshislav@rambler.ru
ORCID iD: https://orcid.org/0009-0004-9704-6896

Ledeneva Tatyana M. — DSc in Technical Sciences, Professor, Department of Computational Mathematics and Applied Information Technologies, Faculty of Applied Mathematics, Informatics and Mechanics, Voronezh State University.
E-mail: ledeneva-tm@yandex.ru
ORCID iD: https://orcid.org/0000-0002-3944-2266